AD23431 - STATISTICAL ANALYSIS AND COMPUTING

EXP 3: Implementation of List, Factors, and Data Frames in R

1. Employee Information Management Using Lists

An organization stores heterogeneous employee details such as **ID, Name, Salary, Department, and Joining Date**.

Using Lists in R, do the following:

a. Create a list to store employee information.

PROGRAM:

```
emp<-list(id=1,name="Ravi",salary=40000,dept="HR",joindate="2022-01-10")
cat("(a)Employee List:\n")
print(emp)
```

OUTPUT:

```
(a)Employee List:
$id
[1] 1

$name
[1] "Ravi"

$salary
[1] 40000

$dept
[1] "HR"

$joindate
[1] "2022-01-10"
```

b. Access and modify the employee salary.

PROGRAM:

```
emp$salary<-45000
cat("(b)Updated Salary:\n")
print(emp$salary)
```

OUTPUT:

```
(b)Updated Salary:
[1] 45000
```

c. Add a new element for **Performance Rating**.

PROGRAM:

```
emp$rating<-"Good"
cat("(c)After Adding Rating:\n")
print(emp)
```

OUTPUT:

```
(c)After Adding Rating:
$id
[1] 1

$name
[1] "Ravi"

$salary
[1] 45000

$dept
[1] "HR"

$joindate
[1] "2022-01-10"

$rating
[1] "Good"
```

d. Remove an element from the list.

PROGRAM:

```
emp$dept<-NULL
cat("(d)After Removing Department:\n")
print(emp)
```

OUTPUT:

```
$id
[1] 1

$name
[1] "Ravi"

$salary
[1] 45000

$joindate
```

```
[1] "2022-01-10"

$rating
[1] "Good"
```

e. Convert the list into a data frame.

PROGRAM:

```
empdf<-as.data.frame(emp)
cat("(e)Converted to Data Frame:\n")
print(empdf)
```

OUTPUT:

```
(e)Converted to Data Frame:
  id name salary   joindate rating
1  1 Ravi  45000 2022-01-10   Good
```

---

2. Student Academic Record Using Nested Lists

A student's academic details include **personal information** and **semester-wise marks**.

Using Lists in R, do the following:

a. Create a nested list containing student details and semester-wise marks.

PROGRAM:

```
stud<-list(
  info=list(name="Shyam",roll=267),
  marks=list(sem1=c(Math=70,Sci=75),sem2 = c(Math=80,Sci=85))
)
cat("(a)Student Nested List:\n")
print(stud)
```

OUTPUT:

```
(a)Student Nested List:
$info
$info$name
[1] "Shyam"

$info$roll
[1] 267


$marks
$marks$sem1
```

```
Math  Sci
  70   75

$marks$sem2
Math  Sci
  80   85
```

b. Extract marks of a specific semester.

PROGRAM:

```
cat("(b) Semester 1 Marks:\n")
print(stud$marks$sem1)
```

OUTPUT:

```
(b) Semester 1 Marks:
Math  Sci
  70   75
```

c. Calculate the average marks across all semesters.

PROGRAM:

```
avg<-mean(c(stud$marks$sem1, stud$marks$sem2))
cat("(c) Average Marks:", avg, "\n")
```

OUTPUT:

```
(c) Average Marks: 77.5
```

d. Update the marks of one subject.

PROGRAM:

```
stud$marks$sem1["Math"]<-90
cat("(d) Updated Semester 1 Marks:\n")
print(stud$marks$sem1)
```

OUTPUT:

```
(d) Updated Semester 1 Marks:
Math  Sci
  90   75
```

e. Display the structure of the list.

PROGRAM:

```
cat("(e)Structure of List:\n")
str(stud)
```

OUTPUT:

```
(e)Structure of List:
List of 2
 $ info :List of 2
  ..$ name: chr "Shyam"
  ..$ roll: num 267
 $ marks:List of 2
  ..$ sem1: Named num [1:2] 90 75
  .. ..- attr(*, "names")= chr [1:2] "Math" "Sci"
  ..$ sem2: Named num [1:2] 80 85
  .. ..- attr(*, "names")= chr [1:2] "Math" "Sci"
```

---

3. Customer Feedback Analysis Using Factors

A café collects customer feedback categorized as **Excellent, Good, Average, and Poor**.

Using Factors in R, do the following:

a. Create a factor variable for customer feedback.

PROGRAM:

```
feed<-factor(c("Good", "Poor", "Excellent", "Average", "Good"))
cat("(a)Feedback Factor:\n")
print(feed)
```

OUTPUT:

```
(a)Feedback Factor:
[1] Good      Poor      Excellent Average   Good
Levels: Average Excellent Good Poor
```

b. Display factor levels and their frequency.

PROGRAM:

```
cat("(b)Levels:\n")
print(levels(feed))
cat("Frequency:\n")
print(table(feed))
```

OUTPUT:

```
(b)Levels:
[1] "Average"   "Excellent" "Good"      "Poor"
Frequency:
feed
  Average Excellent      Good      Poor
        1         1         2         1
```

c. Reorder the factor levels based on sentiment.

PROGRAM:

```
feedord<-factor(feed,levels = c("Poor", "Average", "Good",
"Excellent"),ordered = TRUE)
cat("(c)Reordered Factor:\n")
print(feedord)
```

OUTPUT:

```
(c)Reordered Factor:
[1] Good      Poor      Excellent Average   Good
Levels: Poor < Average < Good < Excellent
```

d. Convert the factor into a numeric scale for analysis.

PROGRAM:

```
cat("(d)Numeric Form:\n")
print(as.numeric(feedord))
```

OUTPUT:

```
(d)Numeric Form:
[1] 3 1 4 2 3
```

e. Check whether the factor is ordered or unordered.

PROGRAM:

```
cat("(e)Is Ordered?:\n")
print(is.ordered(feedord))
```

OUTPUT:

```
(e)Is Ordered?:
[1] TRUE
```

---

4. Employee Shift Scheduling Using Factors

Employees work in **Morning, Evening, and Night** shifts.

Using Factors in R, do the following:

a. Create an ordered factor for work shifts.

PROGRAM:

```
shift<-
factor(c("Morning","Night","Evening","Morning"),levels=c("Morning",
"Evening", "Night"),ordered = TRUE)
cat("(a)Ordered Shift Factor:\n")
```

```
print(shift)
```

OUTPUT:

```
(a)Ordered Shift Factor:
[1] Morning Night   Evening Morning
Levels: Morning < Evening < Night
```

b. Change the reference level of the factor.

PROGRAM:

```
shift<-factor(shift,levels = c("Morning", "Evening", "Night"),ordered =
TRUE)
cat("(b)Reference Level Set to Morning:\n")
print(shift)
```

OUTPUT:

```
(b)Reference Level Set to Morning:
[1] Morning Night   Evening Morning
Levels: Morning < Evening < Night
```

c. Count the number of employees per shift.

PROGRAM:

```
cat("(c)Employee Count per Shift:\n")
print(table(shift))
```

OUTPUT:

```
(c)Employee Count per Shift:
shift
Morning Evening   Night
      2       1       1
```

d. Convert the factor to character type.

PROGRAM:

```
cat("(d)Shift as Character:\n")
print(as.character(shift))
```

OUTPUT:

```
(d)Shift as Character:
[1] "Morning" "Night"   "Evening" "Morning"
```

e. Explain the importance of ordered factors in analysis.

Ordered factors preserve ranking and improve statistical analysis.

5. Student Performance Analysis Using Data Frames

A class maintains records of **Roll No, Name, Marks in Three Subjects, and Result**.

Using Data Frames in R, do the following:

a. Create a data frame for student performance.

PROGRAM:

```
studdf<-data.frame(roll=c(1, 2, 3),name=c("A", "B", "C"),s1=c(75, 80, 65),s2=c(70, 85, 60),s3=c(72, 90, 68))
cat("(a)Student Data Frame:\n")
print(studdf)
```

OUTPUT:

```
(a)Student Data Frame:
  roll name s1 s2 s3
1    1    A 75 70 72
2    2    B 80 85 90
3    3    C 65 60 68
```

b. Add a new column for **Total Marks**.

PROGRAM:

```
studdf$total<-studdf$s1+studdf$s2+studdf$s3
cat("(b)After Adding Total Marks:\n")
print(studdf)
```

OUTPUT:

```
(b)After Adding Total Marks:
  roll name s1 s2 s3 total
1    1    A 75 70 72   217
2    2    B 80 85 90   255
3    3    C 65 60 68   193
```

c. Filter students who scored more than **70** in all subjects.

PROGRAM:

```
cat("(c)Students with >70 in All Subjects:\n")
print(studdf[studdf$s1>70 & studdf$s2>70 & studdf$s3>70,])
```

OUTPUT:

```
(c)Students with >70 in All Subjects:
  roll name s1 s2 s3 total
2    2    B 80 85 90   255
```

d. Sort students based on total marks.

PROGRAM:

```
cat("(d)Sorted by Total Marks:\n")
print(studdf[order(-studdf$total), ])
```

OUTPUT:

```
(d)Sorted by Total Marks:
  roll name s1 s2 s3 total
2    2    B 80 85 90   255
1    1    A 75 70 72   217
3    3    C 65 60 68   193
```

e. Export the data frame to a CSV file.

PROGRAM:

```
write.csv(studdf,"C:/Users/SHYAM/OneDrive/Dokumen/a.csv",row.nam
es = FALSE)
cat("(e) Data exported successfully\n")
```

OUTPUT:

```
(e) Data exported successfully
```

---

6. Sales Dataset Exploration Using Data Frames

A retail store maintains daily sales data including **Date, Product, Quantity, Price, and Total Amount**.

Using Data Frames in R, do the following:

a. Create a data frame for sales data.

PROGRAM:

```
salesdf<-data.frame(product=c("Pen", "Book", "Pen"),qty=c(10, NA,
5),price=c(5, 30, 5))
salesdf$total<-salesdf$qty*salesdf$price
cat("(a)Sales Data:\n")
print(salesdf)
```

OUTPUT:

```
(a)Sales Data:
  product qty price total
1     Pen  10     5    50
2    Book  NA    30    NA
3     Pen   5     5    25
```

b. Compute total sales amount per product.

PROGRAM:

```
cat("(b) Total Sales per Product:\n")
prodsales<-aggregate(total ~ product, salesdf, sum)
print(prodsales)
```

OUTPUT:

```
(b) Total Sales per Product:
  product total
1    Book     0
2     Pen    75
```

c. Identify the best-selling product.

PROGRAM:

```
cat("(c)Best Selling Product:\n")
print(prodsales$product[which.max(prodsales$total)])
```

OUTPUT:

```
(c)Best Selling Product:
[1] "Pen"
```

d. Handle missing values in the dataset.

PROGRAM:

```
salesdf$qty[is.na(salesdf$qty)] <- 0
salesdf$total[is.na(salesdf$total)] <- 0
cat("(d) After Handling Missing Values:\n")
print(salesdf)
```

OUTPUT:

```
(d) After Handling Missing Values:
  product qty price total
1     Pen  10     5    50
2    Book   0    30     0
3     Pen   5     5    25
```

e. Display summary statistics of the data frame.

PROGRAM:

```
cat("(e)Summary Statistics:\n")
print(summary(salesdf))
```

OUTPUT:

(e)Summary Statistics:

```
  product             qty          price          total
 Length:3          Min.   : 0.0   Min.   : 5.00   Min.   : 0.0
 Class :character   1st Qu.: 2.5   1st Qu.: 5.00   1st Qu.:12.5
 Mode  :character   Median : 5.0   Median : 5.00   Median :25.0
                    Mean   : 5.0   Mean   :13.33   Mean   :25.0
                    3rd Qu.: 7.5   3rd Qu.:17.50   3rd Qu.:37.5
                    Max.   :10.0   Max.   :30.00   Max.   :50.0
```